

Modul 1: Fondasi Jaringan Saraf, FNN, Aktivasi, dan Loss

Nama: Ahmad Rizqi

NIM: 122450138

Kelas: RA

```

1 import platform
2 import random
3 import time
4
5 import matplotlib.pyplot as plt
6 import numpy as np
7 import pandas as pd
8 import sklearn
9 import torch
10 from sklearn.datasets import make_moons
11 from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler
14 from torch import nn
15 from torch.utils.data import DataLoader, TensorDataset
16
17 NIM = '122450138'
18 assert NIM.isdigit() and len(NIM) >= 4, 'Isi NIM dengan angka sebelum melanjutkan.'
19 NIM_LAST4 = int(NIM[-4:])
20 SEED = 1000 + NIM_LAST4
21 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
22
23 def seed_everything(seed: int) -> None:
24     random.seed(seed)
25     np.random.seed(seed)
26     torch.manual_seed(seed)
27     if torch.cuda.is_available():
28         torch.cuda.manual_seed_all(seed)
29
30 seed_everything(SEED)
31 pd.set_option('display.precision', 4)
32 environment = {
33     'python': platform.python_version(),
34     'numpy': np.__version__,
35     'pandas': pd.__version__,
36     'sklearn': sklearn.__version__,
37     'torch': torch.__version__,
38     'device': str(DEVICE),
39     'seed': SEED,
40 }
41 environment

```

```

{'python': '3.13.15',
 'numpy': '2.1.3',
 'pandas': '2.2.3',
 'sklearn': '1.6.1',
 'torch': '2.11.0+cpu',
 'device': 'cpu',
 'seed': 1138}

```

B. Neuron dan fungsi aktivasi dengan NumPy - 20 poin

```

1 def sigmoid_np(z: np.ndarray) -> np.ndarray:
2     """Sigmoid stabil yang menghindari overflow untuk z sangat negatif."""
3     z = np.asarray(z)
4     result = np.empty_like(z, dtype=np.result_type(z, np.float64))
5     positive = z >= 0
6     result[positive] = 1.0 / (1.0 + np.exp(-z[positive]))
7     exp_z = np.exp(z[~positive])
8     result[~positive] = exp_z / (1.0 + exp_z)
9     return result
10
11 def relu_np(z: np.ndarray) -> np.ndarray:
12     return np.maximum(0.0, z)
13
14 def leaky_relu_np(z: np.ndarray, alpha: float = 0.01) -> np.ndarray:
15     if alpha < 0:
16         raise ValueError('alpha harus non-negatif.')
17     return np.where(z >= 0, z, alpha * z)

```

```

18
19 def neuron_batch(X: np.ndarray, w: np.ndarray, b: float, activation):
20     X = np.asarray(X)
21     w = np.asarray(w)
22     if X.ndim != 2:
23         raise ValueError(f'X harus 2D (batch, fitur), diperoleh shape {X.shape}.')
24     if w.ndim != 1:
25         raise ValueError(f'w harus 1D (fitur,), diperoleh shape {w.shape}.')
26     if X.shape[1] != w.shape[0]:
27         raise ValueError(f'Jumlah fitur X ({X.shape[1]}) tidak cocok dengan w ({w.shape[0]}).')
28     if np.ndim(b) != 0:
29         raise ValueError('b harus skalar.')
30     if not callable(activation):
31         raise TypeError('activation harus callable.')
32     z = X @ w + b
33     a = np.asarray(activation(z))
34     if a.shape != z.shape:
35         raise ValueError('Keluaran activation harus memiliki shape yang sama dengan z.')
36     return z, a
37
38 # Uji minimum - data uji tidak diubah.
39 X_check = np.array([[2.0, -1.0], [0.0, 3.0], [-2.0, 1.0]])
40 w_check = np.array([0.5, -0.5])
41 z_check, a_check = neuron_batch(X_check, w_check, 0.25, sigmoid_np)
42 assert z_check.shape == (3,)
43 assert a_check.shape == (3,)
44 assert np.all(np.isfinite(a_check))
45 assert np.all((a_check > 0) & (a_check < 1))
46 pd.DataFrame({'z': z_check, 'sigmoid(z)': a_check})

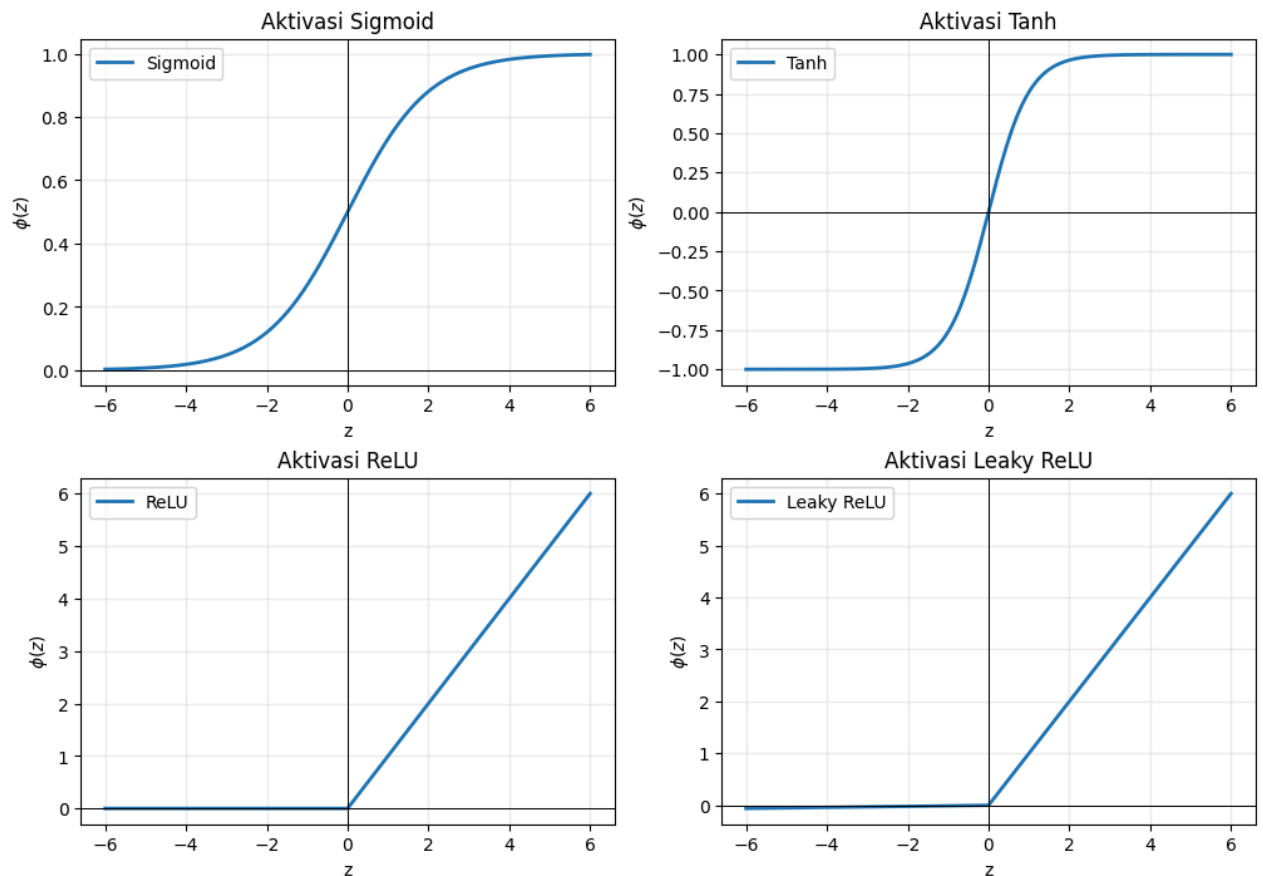
```

	z	sigmoid(z)
0	1.75	0.8520
1	-1.25	0.2227
2	-1.25	0.2227

```

1 grid = np.linspace(-6, 6, 600)
2 activation_values = {
3     'Sigmoid': sigmoid_np(grid),
4     'Tanh': np.tanh(grid),
5     'ReLU': relu_np(grid),
6     'Leaky ReLU': leaky_relu_np(grid),
7 }
8
9 fig, axes = plt.subplots(2, 2, figsize=(10, 7), constrained_layout=True)
10 for ax, (name, values) in zip(axes.ravel(), activation_values.items()):
11     ax.plot(grid, values, linewidth=2, label=name)
12     ax.axhline(0, color='black', linewidth=0.7)
13     ax.axvline(0, color='black', linewidth=0.7)
14     ax.set(title=f'Aktivasi {name}', xlabel='z', ylabel=r'$\phi(z)$')
15     ax.grid(alpha=0.25)
16     ax.legend()
17 plt.show()

```



Interpretasi Fungsi Aktivasi

- Fungsi aktivasi yang berpusat pada nilai nol: tanh, dengan rentang output antara $(-1,1)$. Meskipun Leaky ReLU dapat menghasilkan nilai negatif, distribusi outputnya tidak simetris dan tidak berpusat pada nol seperti fungsi tanh.
- Fungsi aktivasi yang menghasilkan nilai nol pada input negatif: ReLU, yang memberikan output nol ketika menerima masukan bernilai negatif.
- Fungsi aktivasi yang rentan mengalami saturasi: sigmoid dan tanh. Kedua fungsi tersebut dapat mengalami penurunan nilai gradien hingga mendekati nol ketika nilai absolut input semakin besar, sehingga proses pembelajaran jaringan menjadi kurang optimal.
- Keterbatasan fungsi aktivasi linear pada dataset `make_moons`: Penggunaan fungsi aktivasi linear pada beberapa lapisan jaringan tetap menghasilkan transformasi affine tunggal. Akibatnya, model hanya mampu membentuk batas keputusan linear berupa garis, sedangkan dataset `make_moons` memiliki dua kelas dengan pola melengkung yang membutuhkan batas keputusan nonlinier agar dapat dipisahkan dengan baik.

✓ C. Forward pass manual - bagian dari 20 poin pemetaan

Gunakan arsitektur $2 \rightarrow 2 \rightarrow 1$, ReLU pada hidden layer, dan target $y = 1$. Jangan mengubah bobot.

```
1 x_manual = np.array([[2.0, -1.0]], dtype=np.float32)
2 W1 = np.array([[0.5, -0.5], [1.0, 1.0]], dtype=np.float32)
3 b1 = np.array([0.0, 0.0], dtype=np.float32)
4 W2 = np.array([[2.0, -1.0]], dtype=np.float32)
5 b2 = np.array([0.5], dtype=np.float32)
6 y_manual = np.array([1.0], dtype=np.float32)
7
8 z1_np = x_manual @ W1.T + b1
9 h_np = relu_np(z1_np)
10 logit_np = h_np @ W2.T + b2
11 prob_np = sigmoid_np(logit_np)
12 eps = np.finfo(np.float64).eps
13 prob_for_loss = np.clip(prob_np, eps, 1.0 - eps)
14 bce_np = np.mean(
15     -(y_manual * np.log(prob_for_loss) + (1.0 - y_manual) * np.log(1.0 -
16         prob_for_loss))
17 )
```

```

17
18 parameter_count_manual = W1.size + b1.size + W2.size + b2.size
19
20 manual_forward = pd.DataFrame({
21     'besaran': ['z1', 'h', 'logit', 'probabilitas', 'BCE', 'jumlah parameter'],
22     'nilai': [z1_np.tolist(), h_np.tolist(), logit_np.tolist(),
23              prob_np.tolist(), float(bce_np), parameter_count_manual],
24 })
25 display(manual_forward)

```

	besaran	nilai
0	z1	[[1.5, 1.0]]
1	h	[[1.5, 1.0]]
2	logit	[[2.5]]
3	probabilitas	[[0.9241418242454529]]
4	BCE	0.0789
5	jumlah parameter	9

Langkah berikutnya:

[Buat kode dengan manual_forward](#)[New interactive sheet](#)

Tabel shape dan nilai

Tensor	Shape	Nilai
x	(1, 2)	[[2.0, -1.0]]
$W^{(1)}$	(2, 2)	[[0.5, -0.5], [1.0, 1.0]]
$z^{(1)}$	(1, 2)	[[1.5, 1.0]]
h = ReLU ($z^{(1)}$)	(1, 2)	[[1.5, 1.0]]
logit	(1, 1)	[[2.5]]
probabilitas	(1, 1)	[[0.92414182]]
BCE	skalar	0.07888973

Jumlah parameter dan perhitungannya: $W_1 + b_1 + W_2 + b_2 = 4 + 2 + 2 + 1 = 9$ parameter. Secara umum, arsitektur $2 \rightarrow h \rightarrow 1$ mempunyai $(2h + h) + (h + 1) = 4h + 1$ parameter.

✓ D. Pencocokan NumPy dan PyTorch - 20 poin

Salin bobot ke `nn.Sequential`. Jangan menambahkan sigmoid ke model karena loss menerima logit.

```

1 manual_model = nn.Sequential(
2     nn.Linear(2, 2),
3     nn.ReLU(),
4     nn.Linear(2, 1),
5 )
6
7 with torch.no_grad():
8     manual_model[0].weight.copy_(torch.from_numpy(W1))
9     manual_model[0].bias.copy_(torch.from_numpy(b1))
10    manual_model[2].weight.copy_(torch.from_numpy(W2))
11    manual_model[2].bias.copy_(torch.from_numpy(b2))
12
13 x_manual_t = torch.from_numpy(x_manual)
14 y_manual_t = torch.from_numpy(y_manual)
15 torch_logit = manual_model(x_manual_t)
16 torch_probability = torch.sigmoid(torch_logit)
17 torch_loss = nn.BCEWithLogitsLoss()(torch_logit, y_manual_t)
18
19 logit_difference = np.max(np.abs(torch_logit.detach().numpy() - logit_np))
20 loss_difference = abs(torch_loss.item() - float(bce_np))
21 assert logit_difference < 1e-6
22 assert loss_difference < 1e-6
23 assert sum(p.numel() for p in manual_model.parameters()) == parameter_count_manual
24
25 pd.DataFrame({
26     'pemeriksaan': ['selisih logit NumPy-PyTorch', 'selisih BCE NumPy-PyTorch', 'parameter PyTorch'],
27     'nilai': [logit_difference, loss_difference, parameter_count_manual],
28 })

```

	pemeriksaan	nilai
0	selisih logit NumPy-PyTorch	0.0000e+00
1	selisih BCE NumPy-PyTorch	5.3674e-09
2	parameter PyTorch	9.0000e+00

Checkpoint menit ke-85

- Plot empat fungsi aktivasi: **tersedia dan lengkap**.
- Tabel shape dan parameter: **tersedia; total 9 parameter**.
- Selisih forward NumPy-PyTorch: **divalidasi dengan assertion kurang dari 10^{-6}** .

✓ E. Dataset dan protokol eksperimen

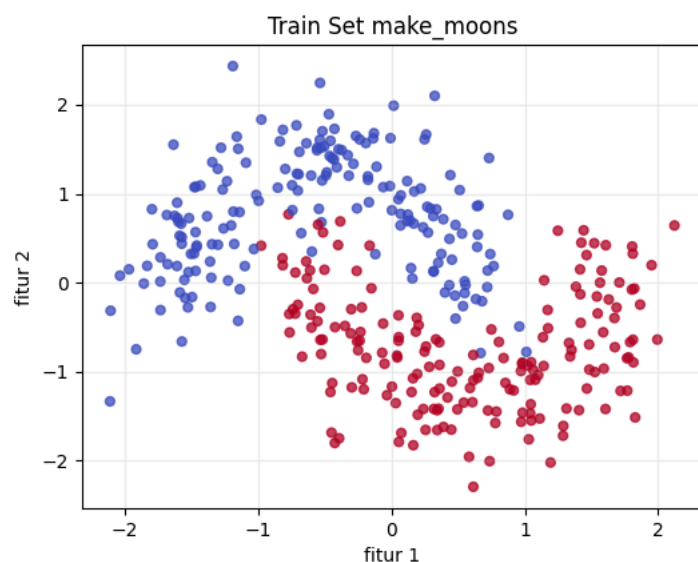
Kode split dan standardisasi diberikan agar fokus tetap pada FNN. Jangan menggunakan test set sampai model final dipilih.

```

1 X, y = make_moons(n_samples=600, noise=0.22, random_state=SEED)
2 X_train, X_temp, y_train, y_temp = train_test_split(
3     X, y, test_size=0.40, stratify=y, random_state=SEED
4 )
5 X_val, X_test, y_val, y_test = train_test_split(
6     X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
7 )
8 scaler = StandardScaler().fit(X_train)
9 X_train_s = scaler.transform(X_train).astype(np.float32)
10 X_val_s = scaler.transform(X_val).astype(np.float32)
11 X_test_s = scaler.transform(X_test).astype(np.float32)
12 y_train_f = y_train.astype(np.float32).reshape(-1, 1)
13 y_val_f = y_val.astype(np.float32).reshape(-1, 1)
14 y_test_f = y_test.astype(np.float32).reshape(-1, 1)
15
16 split_summary = pd.DataFrame({
17     'split': ['train', 'validation', 'test'],
18     'n': [len(y_train), len(y_val), len(y_test)],
19     'positive_rate': [y_train.mean(), y_val.mean(), y_test.mean()],
20 })
21 display(split_summary)
22
23 fig, ax = plt.subplots(figsize=(6, 4.5))
24 ax.scatter(X_train_s[:, 0], X_train_s[:, 1], c=y_train, cmap='coolwarm', s=24,
25     alpha=0.75)
26 ax.set(title='Train Set make_moons', xlabel='fitur 1', ylabel='fitur 2')
27 plt.show()

```

	split	n	positive_rate
0	train	360	0.5
1	validation	120	0.5
2	test	120	0.5



Langkah berikutnya:

[Buat kode dengan split_summary](#)[New interactive sheet](#)

Pemeriksaan Anti-Data Leakage

Scaler hanya dilakukan proses fit menggunakan data training agar nilai rata-rata (mean) dan simpangan baku (standard deviation) yang digunakan dalam standardisasi berasal dari data yang tersedia selama tahap pelatihan. Apabila scaler di-fit pada keseluruhan dataset, informasi dari data validasi dan pengujian dapat memengaruhi proses transformasi fitur. Kondisi tersebut dapat menyebabkan data leakage, sehingga hasil evaluasi model menjadi terlalu optimistis dan tidak merepresentasikan kemampuan generalisasi yang sebenarnya. Setelah parameter scaler diperoleh dari data training, parameter yang sama digunakan untuk mentransformasi data validasi dan data testing tanpa melakukan proses fit ulang.

✓ F. Model dan utilitas training

Lengkapi pemilihan aktivasi dan model. Training loop diberikan; mekanismenya dibahas pada Modul 2.

```

1 def activation_layer(name: str) -> nn.Module:
2     activations = {
3         'relu': nn.ReLU,
4         'tanh': nn.Tanh,
5         'sigmoid': nn.Sigmoid,
6     }
7     normalized = name.strip().lower()
8     if normalized not in activations:
9         raise ValueError(f'Aktivasi {name!r} tidak didukung; pilih {list(activations)}.')
10    return activations[normalized]()
11
12 def build_model(hidden_dim: int, activation: str, seed: int = SEED):
13     if not isinstance(hidden_dim, int) or hidden_dim <= 0:
14         raise ValueError('hidden_dim harus bilangan bulat positif.')
15     seed_everything(seed)
16     model = nn.Sequential(
17         nn.Linear(2, hidden_dim),
18         activation_layer(activation),
19         nn.Linear(hidden_dim, 1),
20     )
21     return model.to(DEVICE)
22
23 def count_parameters(model: nn.Module) -> int:
24     return sum(parameter.numel() for parameter in model.parameters())
25
26 model_check = build_model(8, 'relu')
27 assert count_parameters(model_check) == 4 * 8 + 1 == 33
28 model_check

```

```

Sequential(
  (0): Linear(in_features=2, out_features=8, bias=True)
  (1): ReLU()
  (2): Linear(in_features=8, out_features=1, bias=True)
)

```

```

1 def as_tensor(array):
2     return torch.as_tensor(array, dtype=torch.float32)
3
4 X_train_t, y_train_t = as_tensor(X_train_s), as_tensor(y_train_f)
5 X_val_t, y_val_t = as_tensor(X_val_s).to(DEVICE), as_tensor(y_val_f).to(DEVICE)
6 X_test_t, y_test_t = as_tensor(X_test_s).to(DEVICE), as_tensor(y_test_f).to(
    DEVICE)
7
8 def evaluate(model, X_tensor, y_tensor):
9     model.eval()
10    with torch.no_grad():
11        logits = model(X_tensor)
12        loss = nn.functional.binary_cross_entropy_with_logits(logits, y_tensor).
            item()
13        probabilities = torch.sigmoid(logits)
14        predictions = (probabilities >= 0.5).to(torch.int64)
15        accuracy = (predictions == y_tensor.to(torch.int64)).float().mean().item()
16    return {
17        'loss': loss,
18        'accuracy': accuracy,
19        'probabilities': probabilities.cpu().numpy().ravel(),
20        'predictions': predictions.cpu().numpy().ravel(),
21    }
22
23 def train_model(model, epochs=200, learning_rate=0.05, batch_size=32, seed=SEED)

```

```

24 generator = torch.Generator().manual_seed(seed)
25 loader = DataLoader(
26     TensorDataset(X_train_t, y_train_t),
27     batch_size=batch_size, shuffle=True, generator=generator,
28 )
29 optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
30 criterion = nn.BCEWithLogitsLoss()
31 history = []
32 start = time.perf_counter()
33 for epoch in range(1, epochs + 1):
34     model.train()
35     loss_sum = 0.0
36     for X_batch, y_batch in loader:
37         X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
38         optimizer.zero_grad()
39         logits = model(X_batch)
40         loss = criterion(logits, y_batch)
41         loss.backward()
42         optimizer.step()
43         loss_sum += loss.item() * len(X_batch)
44     val_metrics = evaluate(model, X_val_t, y_val_t)
45     history.append({
46         'epoch': epoch,
47         'train_loss': loss_sum / len(X_train_t),
48         'val_loss': val_metrics['loss'],
49         'val_accuracy': val_metrics['accuracy'],
50     })
51 return pd.DataFrame(history), time.perf_counter() - start

```

✓ G. Baseline dan latihan kelas

Baseline: ReLU, hidden size 8, SGD, learning rate 0.05, batch size 32, dan 200 epoch.

Karena digit terakhir NIM contoh adalah **0**, variasi individual menggunakan **tanh** dengan hidden size 8 dan komponen lain tetap.

Prediksi: Penggunaan fungsi aktivasi tanh diperkirakan dapat menghasilkan nilai validation loss yang sedikit lebih rendah dibandingkan baseline ReLU. Hal ini disebabkan oleh karakteristik tanh yang berpusat pada nilai nol serta bentuk kurvanya yang lebih halus ketika diterapkan pada fitur yang telah melalui proses standardisasi. Namun, perbedaan nilai accuracy belum tentu terlihat karena kedua model masih dapat menghasilkan prediksi kelas yang sama, meskipun tingkat keyakinan atau probabilitas prediksinya berbeda.

```

1 baseline_model = build_model(hidden_dim=8, activation='relu')
2 baseline_history, baseline_seconds = train_model(
3     baseline_model, epochs=200, learning_rate=0.05, batch_size=32,
4 )
5 baseline_train = evaluate(baseline_model, X_train_t.to(DEVICE), y_train_t.to
6     (DEVICE))
7 baseline_val = evaluate(baseline_model, X_val_t, y_val_t)
8 individual_activation = 'tanh' if int(NIM[-1]) <= 4 else 'sigmoid'
9 individual_model = build_model(hidden_dim=8, activation=individual_activation)
10 individual_history, individual_seconds = train_model(
11     individual_model, epochs=200, learning_rate=0.05, batch_size=32,
12 )
13 individual_train = evaluate(individual_model, X_train_t.to(DEVICE), y_train_t.to
14     (DEVICE))
15 individual_val = evaluate(individual_model, X_val_t, y_val_t)
16 class_exercise_metrics = pd.DataFrame([
17     {
18         'run': 'baseline_relu_h8', 'train_loss': baseline_train['loss'],
19         'train_accuracy': baseline_train['accuracy'], 'val_loss': baseline_val
20         ['loss'],
21         'val_accuracy': baseline_val['accuracy'], 'duration_seconds':
22         baseline_seconds,
23     },
24     {
25         'run': f'individual_{individual_activation}_h8', 'train_loss':
26         individual_train['loss'],
27         'train_accuracy': individual_train['accuracy'], 'val_loss':
28         individual_val['loss'],
29         'val_accuracy': individual_val['accuracy'], 'duration_seconds':
30         individual_seconds,
31     }
32 ])
33 display(class_exercise_metrics)
34 fig, ax = plt.subplots(figsize=(8, 5))
35 ax.plot(baseline_history['epoch'], baseline_history['train_loss'], label='ReLU

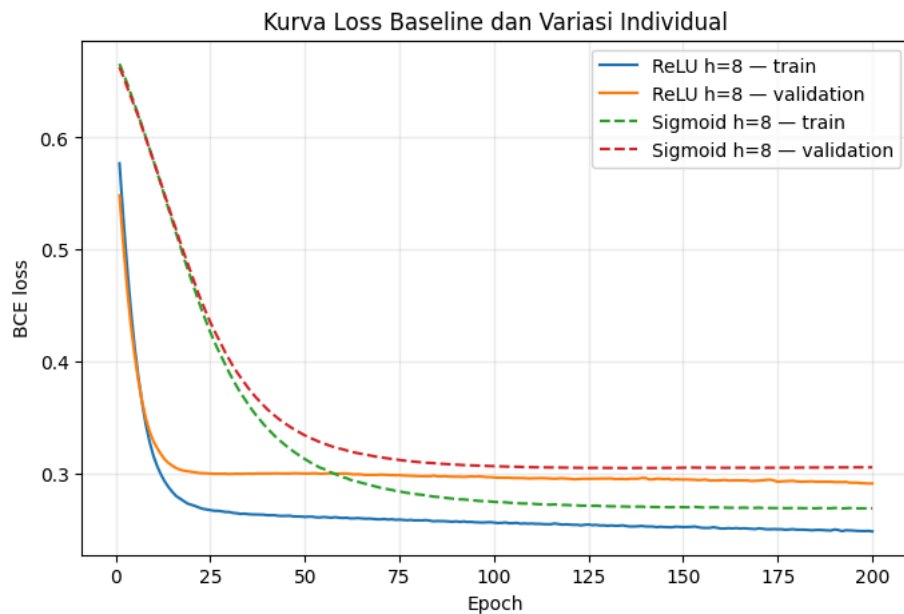
```

```

h=8 - train')
32 ax.plot(baseline_history['epoch'], baseline_history['val_loss'], label='ReLU
h=8 - validation')
33 ax.plot(individual_history['epoch'], individual_history['train_loss'], '--',
34         label=f'{individual_activation.title()} h=8 - train')
35 ax.plot(individual_history['epoch'], individual_history['val_loss'], '--',
36         label=f'{individual_activation.title()} h=8 - validation')
37 ax.set(title='Kurva Loss Baseline dan Variasi Individual', xlabel='Epoch',
        ylabel='BCE loss')
38 ax.grid(alpha=0.25)
39 ax.legend()
40 plt.show()

```

	run	train_loss	train_accuracy	val_loss	val_accuracy	duration_seconds
0	baseline_relu_h8	0.2476	0.8972	0.2909	0.8667	1.7547
1	individual_sigmoid_h8	0.2686	0.8833	0.3054	0.8583	2.6424



Langkah berikutnya: [Buat kode dengan class_exercise_metrics](#) [New interactive sheet](#)

✓ H. Tugas individual: enam eksperimen - 20 poin

Jalankan kombinasi aktivasi (`{relu, tanh, sigmoid}`) dan hidden size (`{4, 16}`). Gunakan validation loss untuk memilih model.

```

1 ACTIVATIONS = ['relu', 'tanh', 'sigmoid']
2 HIDDEN_SIZES = [4, 16]
3 EPOCHS = 200
4 LEARNING_RATE = 0.05
5 BATCH_SIZE = 32
6
7 experiment_rows = []
8 trained_models = {}
9 histories = {}
10
11 for activation in ACTIVATIONS:
12     for hidden_dim in HIDDEN_SIZES:
13         run_id = f'{activation}_h{hidden_dim}'
14         model = build_model(hidden_dim, activation, seed=SEED)
15         history, duration = train_model(
16             model,
17             epochs=EPOCHS,
18             learning_rate=LEARNING_RATE,
19             batch_size=BATCH_SIZE,
20             seed=SEED,
21         )
22         train_metrics = evaluate(model, X_train_t.to(DEVICE), y_train_t.to(DEVICE))
23         val_metrics = evaluate(model, X_val_t, y_val_t)
24
25         trained_models[run_id] = model
26         histories[run_id] = history
27         experiment_rows.append({
28             'run_id': run_id,
29             'activation': activation,
30             'hidden_dim': hidden_dim,

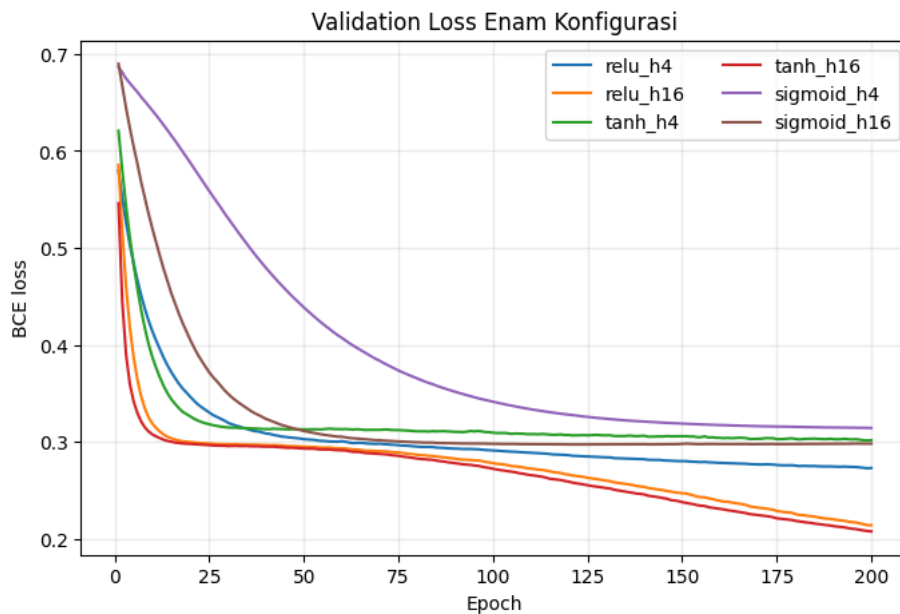
```

```

31     'parameters': count_parameters(model),
32     'epochs': EPOCHS,
33     'learning_rate': LEARNING_RATE,
34     'batch_size': BATCH_SIZE,
35     'train_loss': train_metrics['loss'],
36     'train_accuracy': train_metrics['accuracy'],
37     'val_loss': val_metrics['loss'],
38     'val_accuracy': val_metrics['accuracy'],
39     'duration_seconds': duration,
40     'seed': SEED,
41 })
42
43 results = pd.DataFrame(experiment_rows).sort_values('val_loss').reset_index(drop=True)
44 assert len(results) == len(ACTIVATIONS) * len(HIDDEN_SIZES) == 6
45 assert results['run_id'].is_unique
46 display(results)
47 results.to_csv(f'M01_{NIM_LAST4:04d}_metrics.csv', index=False)
48
49 fig, ax = plt.subplots(figsize=(8, 5))
50 for run_id, history in histories.items():
51     ax.plot(history['epoch'], history['val_loss'], label=run_id)
52 ax.set(title='Validation Loss Enam Konfigurasi', xlabel='Epoch', ylabel='BCE loss')
53 ax.grid(alpha=0.25)
54 ax.legend(ncol=2)
55 plt.show()

```

	run_id	activation	hidden_dim	parameters	epochs	learning_rate	batch_size	train_loss	train_accuracy	val_loss
0	tanh_h16	tanh	16	65	200	0.05	32	0.1567	0.9444	0.2077
1	relu_h16	relu	16	65	200	0.05	32	0.1651	0.9417	0.2142
2	relu_h4	relu	4	17	200	0.05	32	0.2411	0.8889	0.2731
3	sigmoid_h16	sigmoid	16	65	200	0.05	32	0.2648	0.8889	0.2982
4	tanh_h4	tanh	4	17	200	0.05	32	0.2663	0.8917	0.3017
5	sigmoid_h4	sigmoid	4	17	200	0.05	32	0.2796	0.8889	0.3144



Langkah berikutnya:

[Buat kode dengan results](#)[New interactive sheet](#)

✓ I. Evaluasi model final - 15 poin

Pilih satu model dengan validation loss terendah. Baru setelah itu evaluasi test set satu kali.

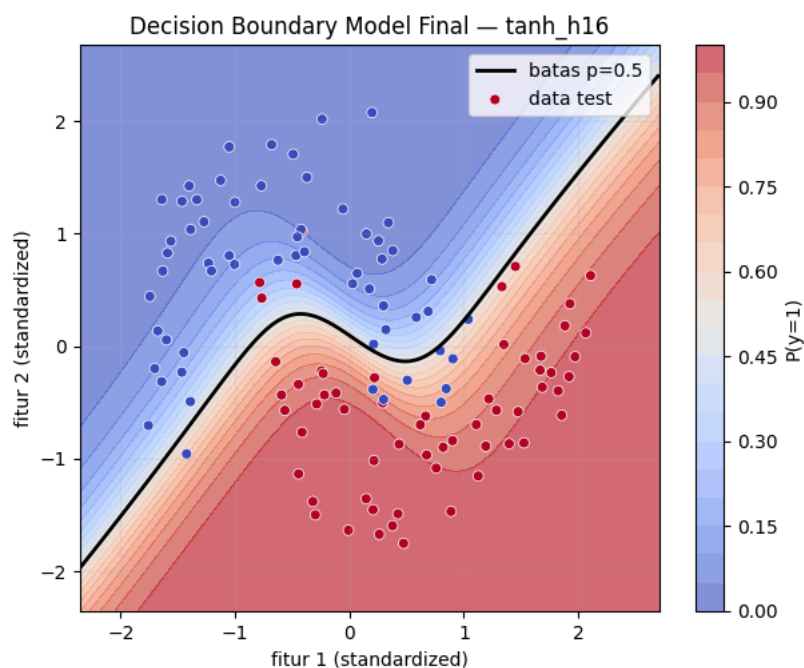
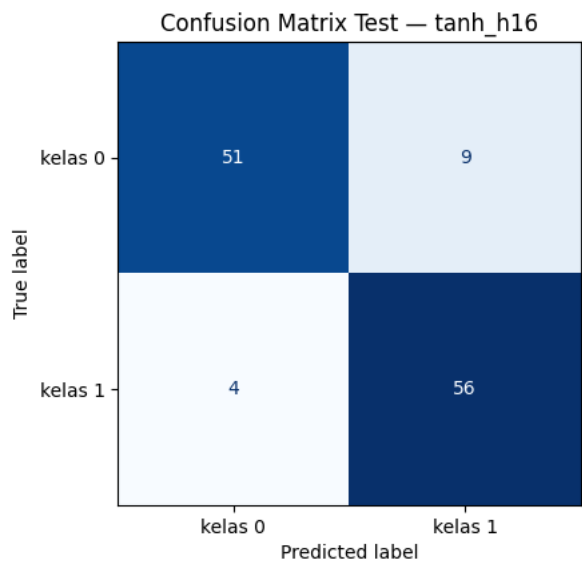
```

1 def plot_decision_boundary(model, X_values, y_values, title):
2     x_min, x_max = X_values[:, 0].min() - 0.6, X_values[:, 0].max() + 0.6
3     y_min, y_max = X_values[:, 1].min() - 0.6, X_values[:, 1].max() + 0.6
4     xx, yy = np.meshgrid(
5         np.linspace(x_min, x_max, 300),
6         np.linspace(y_min, y_max, 300),
7     )
8     mesh = np.column_stack([xx.ravel(), yy.ravel()]).astype(np.float32)
9     model.eval()

```

```
10 with torch.no_grad():
11     probabilities = torch.sigmoid(model(as_tensor(mesh).to(DEVICE)))
12     probability_grid = probabilities.cpu().numpy().reshape(xx.shape)
13
14 fig, ax = plt.subplots(figsize=(7, 5.5))
15 filled = ax.contourf(xx, yy, probability_grid, levels=np.linspace(0, 1, 21),
16                     cmap='coolwarm', alpha=0.65)
17 ax.contour(xx, yy, probability_grid, levels=[0.5],
18           colors='black', linewidths=2)
19 # Proxy artist menjaga legenda kompatibel dengan berbagai versi Matplotlib.
20 ax.plot([], [], color='black', linewidth=2, label='batas p=0.5')
21 ax.scatter(X_values[:, 0], X_values[:, 1], c=y_values, cmap='coolwarm',
22           edgecolor='white', linewidth=0.5, s=32, label='data test')
23 ax.set(title=title, xlabel='fitur 1 (standardized)', ylabel='fitur 2 (standardized)')
24 ax.grid(alpha=0.2)
25 ax.legend()
26 fig.colorbar(filled, ax=ax, label='P(y=1)')
27 plt.show()
28
29 best_run_id = results.loc[0, 'run_id']
30 best_model = trained_models[best_run_id]
31 # Inilah satu-satunya evaluasi test pada alur notebook.
32 test_metrics = evaluate(best_model, X_test_t, y_test_t)
33 final_metrics = pd.DataFrame([
34     'run_id': best_run_id,
35     'selection_val_loss': results.loc[0, 'val_loss'],
36     'test_loss': test_metrics['loss'],
37     'test_accuracy': test_metrics['accuracy'],
38 ])
39 display(final_metrics)
40
41 cm = confusion_matrix(y_test, test_metrics['predictions'])
42 fig, ax = plt.subplots(figsize=(5.5, 4.5))
43 ConfusionMatrixDisplay(cm, display_labels=['kelas 0', 'kelas 1']).plot(
44     ax=ax, cmap='Blues', colorbar=False,
45 )
46 ax.set_title(f'Confusion Matrix Test - {best_run_id}')
47 plt.show()
48
49 plot_decision_boundary(
50     best_model,
51     X_test_s,
52     y_test,
53     title=f'Decision Boundary Model Final - {best_run_id}',
54 )
```

	run_id	selection_val_loss	test_loss	test_accuracy
0	tanh_h16	0.2077	0.234	0.8917



✓ J. Analisis dan refleksi - 10 poin

Analisis di bawah dibentuk dari tabel hasil agar tetap benar jika NIM/seed diganti.

```

1 from IPython.display import Markdown
2
3 by_activation = results.set_index(['activation', 'hidden_dim'])
4 best_row = results.iloc[0]
5 worst_row = results.iloc[-1]
6
7 hidden_comparisons = []
8 all_larger_hidden_improved = True
9 for activation in ACTIVATIONS:
10     loss_h4 = by_activation.loc[(activation, 4), 'val_loss']
11     loss_h16 = by_activation.loc[(activation, 16), 'val_loss']
12     direction = 'lebih rendah' if loss_h16 < loss_h4 else 'lebih tinggi atau sama'
13     all_larger_hidden_improved &= loss_h16 < loss_h4
14     hidden_comparisons.append(
15         f"{activation}: h=4 {loss_h4:.4f}, h=16 {loss_h16:.4f} ({direction} pada h=16)"
16     )
17
18 hidden_conclusion = (
19     'Pada eksperimen seed ini, ya: h=16 menurunkan validation loss pada ketiga

```

```

    aktivasi.'
20 'Namun, ini bukan jaminan umum bahwa hidden size yang lebih besar selalu
    lebih baik.'
21 if all_larger_hidden_improved else
22 'Tidak pada eksperimen ini; setidaknya satu aktivasi tidak membaik ketika
    hidden size diperbesar.'
23 )
24
25 stability = (
26     results.groupby('activation')['val_loss']
27     .agg(['min', 'max'])
28     .assign(range=lambda frame: frame['max'] - frame['min'])
29     .sort_values('range')
30 )
31 most_stable = stability.index[0]
32
33 similar_accuracy_pair = None
34 for i in range(len(results)):
35     for j in range(i + 1, len(results)):
36         acc_gap = abs(results.loc[i, 'val_accuracy'] - results.loc[j,
            'val_accuracy'])
37         loss_gap = abs(results.loc[i, 'val_loss'] - results.loc[j, 'val_loss'])
38         candidate = (loss_gap, i, j, acc_gap)
39         if acc_gap <= 0.01 and (similar_accuracy_pair is None or candidate >
            similar_accuracy_pair):
40             similar_accuracy_pair = candidate
41
42 if similar_accuracy_pair is None:
43     closest = []
44     for i in range(len(results)):
45         for j in range(i + 1, len(results)):
46             closest.append((abs(results.loc[i, 'val_accuracy'] - results.loc[j,
                'val_accuracy']), i, j))
47     _, pair_i, pair_j = min(closest)
48 else:
49     _, pair_i, pair_j, _ = similar_accuracy_pair
50
51 pair_a, pair_b = results.loc[pair_i], results.loc[pair_j]
52 hypothesis_matches = best_row['activation'] == 'tanh' and best_row
    ['hidden_dim'] == 16
53 hypothesis_text = (
54     'sesuai' if hypothesis_matches else
55     f"tidak sesuai; pemenangnya {best_row['run_id']}, bukan tanh_h16"
56 )
57
58
59 analysis_markdown = f"""
60 1. **Apakah peningkatan hidden size selalu menghasilkan validation loss yang
    lebih rendah?**
61     {'; '.join(hidden_comparisons)}. {hidden_conclusion}
62
63 2. **Fungsi aktivasi manakah yang menunjukkan kestabilan terbaik pada kedua
    ukuran hidden size?**
64     **{most_stable}**. Aktivasi ini memiliki perbedaan validation loss paling
    kecil antara h=4 dan h=16, dengan rentang sebesar **{stability.loc
    [most_stable, 'range']:.4f}**.
65
66 3. **Apakah terdapat konfigurasi dengan nilai accuracy yang hampir sama, tetapi
    nilai BCE berbeda? Apa penyebabnya?**
67     Sebagai contoh, konfigurasi **{pair_a['run_id']}** menghasilkan accuracy **
    {pair_a['val_accuracy']:.4f}** dan BCE **{pair_a['val_loss']:.4f}**,
    sedangkan konfigurasi **{pair_b['run_id']}** memiliki accuracy **{pair_b
    ['val_accuracy']:.4f}** dan BCE **{pair_b['val_loss']:.4f}**. Perbedaan ini
    terjadi karena accuracy hanya mengevaluasi ketepatan klasifikasi berdasarkan
    ambang probabilitas 0,5, sementara BCE turut mempertimbangkan tingkat
    keyakinan probabilitas yang dihasilkan model. Prediksi yang salah dengan
    tingkat keyakinan tinggi akan memperoleh penalti yang lebih besar pada BCE.
68
69 4. **Pada wilayah mana decision boundary menunjukkan tingkat ketidakpastian
    tertinggi?**
70     Ketidakpastian paling tinggi terdapat pada area pita warna netral di sekitar
    kontur **$p=0.5$**, khususnya pada bagian ketika kedua pola bulan sabit
    saling berdekatan atau mengalami tumpang tindih akibat noise. Di wilayah
    tersebut, nilai probabilitas berada di sekitar 0,5, sehingga perubahan kecil
    pada output model dapat menyebabkan perubahan kelas prediksi

```

